

Analyse syntaxique et fonctions sémantiques.

Dans les premiers exercices qui vont suivre, on traite des arbres binaires ; aux feuilles de ces arbres sont attachées des valeurs entières. On décide de noter $(A.B)$ l'arbre binaire composé des sous-arbres A et B . Si un arbre est réduit à une feuille, il est noté par sa valeur. S'il est vide, il est noté par *nil*.

Ainsi, la notation $((1.2).(3.nil))$ correspond à l'arbre suivant (figure 1) ;

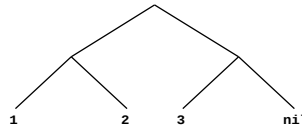


Figure 1: Arbre $((1.2).(3.nil))$

On traite également des listes. On décide de noter une liste $(A, B, \dots, E)$ où $A, B, \dots, E$ sont les éléments successifs de la liste. Les éléments peuvent être des listes ou des arbres, et donc aussi des valeurs entières ou *nil*. Ainsi, $((3.2), nil, nil, 4)$ est une liste.

Pour avoir une représentation commune aux arbres et listes, on décide que la liste $(A, B, C, \dots, E)$ sera représentée par l'arbre $(A.(B.(\dots (E.nil) \dots)))$. La figure 2 illustre la représentation de listes.

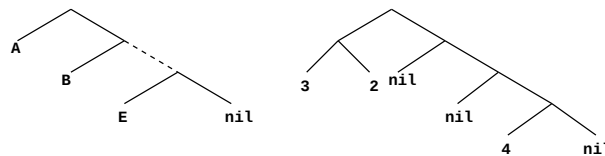


Figure 2: Arbre $(A, B, \dots, E)$ et Arbre $((3.2), nil, nil, 4)$

Analyse syntaxique descendante.

Question 1 : Proposer une grammaire décrivant la syntaxe des arbres et listes. Des exemples de notations correctes sont donnés ci-dessous :

$(1, (2.2), 3)$
 nil
 23
 $((2.3), (2, (3.4), nil))$
 (1)

Question 2 : Peut-on réaliser un analyseur LL(1) pour la grammaire présentée à la question précédente ? Définir l'analyseur syntaxique dans un premier temps au moyen de tables d'analyse, puis à l'aide de fonctions récursives.

Question 3 : Dans l'analyseur LL(1) ainsi construit, ajouter des fonctions sémantiques permettant de construire la notation binaire d'une phrase du langage, c'est à dire avec la notation pointée uniquement.

Analyse syntaxique ascendante.

Exercice 1 : Peut-on réaliser un analyseur SLR(1) pour la grammaire présentée à l'exercice précédent ? Dérouler l'analyse ascendante sur l'exemple suivant : $((1.2), 3)\$$

Exercice 2 : Soit la grammaire suivante dans laquelle l'alphabet terminal est constitué des symboles de l'ensemble $\{= *idf\}$, et S' étant l'axiome augmenté :

S'	$\longrightarrow$	S	r_0
S	$\longrightarrow$	$G = D$	r_1
		D	r_2
G	$\longrightarrow$	$* D$	r_3
		idf	r_4
D	$\longrightarrow$	G	r_5

Peut-on construire des analyseurs SLR(1), LR(1) et LALR(1) ?

Fonctions sémantiques en analyse syntaxique ascendante.

Exercice 1 : Retour sur la grammaire des arbres et listes.

Reprendre la grammaire des arbres et listes définie à la section précédente, et ajouter les fonctions sémantiques permettant de construire la notation binaire d'une phrase du langage.

Même question sur la grammaire récursive gauche suivante :

S'	$\longrightarrow$	A	r_0
A	$\longrightarrow$	V	r_1
		(C)	r_2
C	$\longrightarrow$	$A . A$	r_3
		L	r_4
L	$\longrightarrow$	A	r_6
		L, A	r_7
V	$\longrightarrow$	$entier$	r_8
		nil	r_9

Exercice 2 : Mots de Dyck.

En théorie des langages, les langages de Dyck sont des langages formels particuliers : un langage de Dyck est l'ensemble des *mots bien parenthésés*. Intuitivement, un mot de Dyck est un mot bien parenthésé, c'est à dire qu'il peut être réduit au mot vide en supprimant successivement des paires adjacentes de parenthèses. Par exemple, $(())(())(())$ est un mot de Dyck.

Les langages de Dyck sont des langages algébriques déterministes (langages reconnus par un automate à pile). On donne ci-dessous une grammaire qui reconnaît les mots de Dyck.

S'	$\longrightarrow$	S	r_0
S	$\longrightarrow$	$E S$	r_1
		E	r_2
E	$\longrightarrow$	(S)	r_3
		$(\wedge)$	r_4

Question 1 : Ecrire les fonctions sémantiques qui retournent le nombre de parenthèses non imbriquées. Pour le mot suivant $(())(())(())$ ce nombre vaut 3.

Question 2 : Ecrire les fonctions sémantiques qui retournent le nombre maximal d'imbrication des parenthèses. Pour le mot suivant $(())(())(())$ ce nombre vaut 2.

Exercice 3 : Nombres binaires.

Soit la grammaire suivante :

N	$\longrightarrow$	L	r_0
		$L . L$	r_1
L	$\longrightarrow$	B	r_2
		$B L$	r_3
B	$\longrightarrow$	0	r_4
		1	r_5

Quelle est la valeur en base 10 du nombre binaire 1011.1101 ?

Introduire des fonctions sémantiques qui permettent de calculer la valeur décimale d'un nombre binaire.

On pourra commencer par écrire ces fonctions sémantiques en supposant une analyse syntaxique descendante.